QuantSim

ML-Based Market Simulation & Portfolio Risk Analytics Platform

FYP-1 Mid Report

Project Team

Hamza Kaleem Sheikh — 23i-0783

Ali Mazhar — 23i-0796

Afnan Rizwan — 23i-0561

Session 2023–2027

Supervised by

Dr. Muhammad Rizwan

Co-Supervised by

Dr. Ahmad Raza Shahid

FAST School of Computing
National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *QuantSim: ML-Based Market Simulation and Portfolio Risk Analytics Platform*.

Student 1: Hamza Kaleem Sheikh
Signature:

Student 2: Ali Mazhar
Signature:

Student 3: Afnan Rizwan
Signature:

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, datasets, models, APIs, and relevant sources are disclosed where applicable.

Abstract

QuantSim is an ML-Based Market Simulation and Portfolio Risk Analytics Platform designed to provide a controlled environment for evaluating trading strategies and studying portfolio behavior under varying financial-market conditions. The system combines historical market replay, stochastic market simulation, event-driven trading, portfolio risk analytics, Topological Data Analysis (TDA), and machine-learning-based analysis within a connected research environment. The project is motivated by the difficulty and risk of testing trading strategies under real market conditions, particularly during adverse scenarios that cannot be reproduced on demand. During the current FYP-1 stage, the team has established the research basis, system scope, architectural direction, and initial implementation plan. The remainder of FYP-1 focuses on developing the foundational data and system infrastructure, followed by core simulation, trading, and analytical components.

Acknowledgments

The project team acknowledges the guidance and support provided by Dr. Muhammad Rizwan and Dr. Ahmad Raza Shahid throughout the proposal and development process. The team also acknowledges the faculty of the FAST School of Computing for providing the academic environment and resources required to undertake the project.

Executive Summary

QuantSim is an ML-Based Market Simulation and Portfolio Risk Analytics Platform intended to address the difficulty of safely and systematically evaluating trading strategies and portfolio behavior under changing financial market conditions. Testing strategies using real capital introduces direct financial risk, while conventional historical backtesting cannot arbitrarily generate market conditions such as volatility shocks, liquidity disruptions, or sector-wide deterioration.

The proposed system provides a controlled virtual market in which historical market conditions can be replayed and synthetic or adverse scenarios can be generated. An event-driven trading engine processes orders and updates portfolio state while accounting for transaction costs and slippage. The resulting portfolio behavior is analyzed through conventional financial risk metrics, Topological Data Analysis, and machine-learning techniques.

The project was evaluated against existing financial systems including SimCorp One, QuantConnect, QuantRocket, and AlgoTrader, as well as related frameworks such as FinRL. The investigation showed substantial existing capability in algorithmic trading, backtesting, and portfolio management. QuantSim therefore does not claim novelty in these individual capabilities. Instead, its contribution is centered on integrating controlled market simulation, portfolio-risk analysis, topology-based market-structure analysis, and explainable analytical outputs within one research-oriented environment.

The current development strategy divides the project into four iterations. Iteration 1 establishes the foundations through market-data ingestion, database design, continuous integration, and authentication. Iteration 2 develops the stochastic simulator, event-driven trading engine, and TDA pipeline. Iteration 3 introduces the machine-learning and alerting layers, while Iteration 4 focuses on dashboard refinement, integration testing, and selected extended modules.

List of Abbreviations

Abbreviation	Meaning
FYP	Final Year Project
ML	Machine Learning
TDA	Topological Data Analysis
CCP	Complex Computing Problem
API	Application Programming Interface
NLP	Natural Language Processing
CI	Continuous Integration
OHLCV	Open, High, Low, Close, Volume
P&L	Profit and Loss
SHAP	SHapley Additive exPlanations
GBM	Geometric Brownian Motion
OU	Ornstein–Uhlenbeck

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations vii

1 Introduction and Project Vision 1

1.1 Project Overview 1
1.2 Problem Statement 1
1.3 Problem Context and Motivation 2
1.4 Project Vision 3
1.5 Project Objectives 3
1.6 Project Scope 4
1.6.1 Core Scope 4
1.6.2 Extended Scope 4
1.6.3 Out of Scope 4
1.7 Project Modules 5
1.7.1 Market Data Ingestion 5
1.7.2 Database and Data Management 5
1.7.3 Market Simulation 5
1.7.4 Event-Driven Trading Engine 5
1.7.5 Portfolio Risk Analytics 6
1.7.6 Topological Analysis 6
1.7.7 Machine Learning 6
1.7.8 Risk and Alert Engine 6
1.7.9 Analytics Dashboard 6

1.7.10 Explainability Layer 6
1.8 Stakeholders and User Classes 6
1.8.1 Financial Analysts 6
1.8.2 Quantitative Researchers 7
1.8.3 Day Traders and Strategy Developers 7
1.8.4 Faculty and Project Evaluators 7
1.9 Success Criteria 7
1.10 Constraints and Assumptions 8
1.11 Ethical, Legal and DEI Considerations 8

2 Research and Existing Solutions 9
2.1 Research Method / Approach 9
2.2 Existing Solutions / Products 9
2.3 Comparison of Existing Solutions 10
2.4 Limitations and Gaps 11
2.5 Proposed Solution 12
2.6 Complex Computing Problem 12
2.7 Technical Challenges 13
2.8 Research Findings and Design Decisions 13

3 Proposal Feedback and Iteration Plan 14
3.1 Proposal Feedback Summary 14
3.2 Proposal Action Register 14
3.3 Iteration Strategy 15
3.4 Iteration 1 Plan 16
3.5 Iteration 2 Plan 16
3.6 Work Breakdown and Responsibilities 16
3.7 Progress Against Plan 17
3.8 Delays, Deviations and Recovery Plan 17

4 Software Requirements 18
4.1 User Interactions / Use Cases 18
4.2 Use Case Diagram 18
4.3 Detailed Use Cases 18

4.3.1 Run Historical Market Replay . . . 18
4.3.2 Run Simulated Market Scenario . . . 19
4.3.3 Analyze Portfolio Risk . . . 19
4.3.4 Inspect Structural Market Risk . . . 19
4.4 Functional Requirements . . . 19
4.4.1 Market Data . . . 19
4.4.2 Market Simulation . . . 20
4.4.3 Trading and Portfolio . . . 20
4.4.4 Risk and Analysis . . . 20
4.4.5 Visualization . . . 21
4.5 Non-Functional Requirements . . . 21
4.5.1 Reliability . . . 21
4.5.2 Usability . . . 21
4.5.3 Performance . . . 21
4.5.4 Security . . . 21
4.6 Acceptance Criteria . . . 21
4.7 Requirements Traceability . . . 22

5 System Design **22**
5.1 System Architecture . . . 22
5.2 Architectural Components . . . 22
5.3 Interfaces and Dependencies . . . 23
5.4 Architecture Decisions and Rationale . . . 23
5.5 Design Evolution . . . 24
5.6 Domain Model . . . 24
5.7 Data Design . . . 25
5.8 Process / Behavior Design . . . 25

6 Iteration 1: Design and Implementation **26**
6.1 Iteration Objectives . . . 26
6.2 Selected Use Cases / Features . . . 26
6.3 Design . . . 27
6.4 Implementation . . . 27

6.5 Completed Features / Module 27
6.6 Technical Implementation Details 27
6.7 Algorithms and Technical Decisions 28
6.8 Tools, Frameworks and Technologies 28
6.9 Demonstration Evidence 28
6.10 Iteration Outcome 29

7 Testing and Early Validation 29
7.1 Testing Strategy 29
7.2 Test Levels and Techniques 29
7.3 Test Cases 30
7.4 Test Results 30
7.5 Acceptance Testing / Validation 30
7.6 Edge and Failure Cases 31
7.7 Defects and Corrective Actions 31
7.8 Current Validation Status 31

8 Project Management and Development Evidence 31
8.1 Work Completed 31
8.2 Work Remaining 31
8.3 Individual Responsibilities 32
8.4 Contribution of Each Team Member 32
8.5 Repository and Version Control Evidence 32
8.6 Milestones and Commit Evidence 32
8.7 Tools and Tool-Assisted Development 32
8.8 External Components and Dependencies 33
8.9 Risks and Mitigation 33
8.10 Security and Privacy Considerations 33
8.11 Current Limitations 34

9 Remaining FYP Plan 34
9.1 Remaining Scope 34
9.2 Remaining Modules and Features 35
9.3 Remaining Iterations 35

9.3.1 Iteration 2 35
9.3.2 Iteration 3 35
9.3.3 Iteration 4 35
9.4 Implementation Plan 35
9.5 Testing and Validation Plan 36
9.6 Deployment Plan 36
9.7 Tentative Timeline 37
9.8 Expected Final Deliverables 37

10 Conclusion **38**

A Proposal Action Register **39**

B Detailed Requirements **39**

C Detailed Test Cases **39**

D Repository and Contribution Evidence **39**

E Additional Design Diagrams **39**

F Meeting and Progress Evidence **39**

List of Figures

1 QuantSim Use Case Diagram 18
2 QuantSim System Architecture 22
3 Initial QuantSim Database Design 25
4 QuantSim End-to-End Processing Flow 26
5 Iteration 1 Design 27
6 Iteration 1 Demonstration Evidence 29

List of Tables

2 Comparison Criteria for Existing Solutions 11
3 Proposal Action Register 14
4 Progress Against Planned Milestones 17
5 Iteration 1 Feature Status 27
6 Iteration 1 Technologies 28
7 Representative Test Cases 30
8 Team Responsibilities 32
9 Tentative Development Timeline 37

1 Introduction and Project Vision

1.1 Project Overview

QuantSim is an ML-Based Market Simulation and Portfolio Risk Analytics Platform designed to provide a controlled environment for studying financial market behavior, executing trading strategies, and analyzing the resulting portfolio risk.

The system combines historical market replay with configurable market simulation, an event-driven trading engine, portfolio analytics, machine learning, and Topological Data Analysis (TDA) within a single analytical pipeline.

The primary purpose of QuantSim is to allow users to evaluate how a trading strategy and its resulting portfolio behave under both ordinary and adverse market conditions without exposing real capital to those conditions. Historical market data can be replayed to reproduce previously observed environments, while stochastic and stress-based simulation can be used to construct controlled scenarios.

QuantSim is intended as a research and experimentation environment rather than a live trading system. It will not execute real financial transactions or provide investment advice.

1.2 Problem Statement

Trading strategies are difficult and risky to test under real financial market conditions. Testing a strategy with actual capital exposes users to financial losses when the strategy performs poorly. Furthermore, market conditions that are particularly useful for evaluating a strategy, such as crashes, sudden volatility increases, liquidity shocks, or correlated asset failures, cannot reliably be produced on demand in real markets.

Historical backtesting partially addresses this problem by allowing a strategy to be evaluated against previously observed market data. However, historical replay is inherently limited to the conditions represented by the selected

historical period. It cannot directly provide arbitrary controlled scenarios in which specific market characteristics can be varied.

QuantSim addresses this problem by providing a controlled virtual market in which historical conditions can be replayed and synthetic or stressed conditions can be generated. Trading activity can then be executed against those conditions and the resulting portfolio behavior can be analyzed using financial risk metrics, machine learning, and topology-based methods.

1.3 Problem Context and Motivation

Financial markets are dynamic systems in which relationships between assets, volatility, liquidity, and portfolio exposure can change over time. A trading strategy that performs satisfactorily in one market regime may behave very differently when market conditions change.

Conventional portfolio dashboards primarily describe observed performance through metrics such as return, volatility, and drawdown. While these metrics remain important, they do not necessarily explain structural changes in the market or indicate how a portfolio may behave under deliberately constructed adverse scenarios.

Quantitative researchers also frequently rely on separate tools for data acquisition, backtesting, simulation, machine learning, and mathematical analysis. The separation between these tools can make it difficult to investigate the same financial scenario using multiple analytical methods.

QuantSim is motivated by the opportunity to provide one connected environment in which a market condition can be generated or replayed, a trading strategy can operate within that environment, the resulting portfolio state can be recorded, and multiple analytical methods can be applied to the same experiment.

1.4 Project Vision

The vision of QuantSim is to provide a controlled and auditable environment for financial experimentation in which users can simulate market behavior, execute trading strategies, observe portfolio evolution, and analyze risk using established financial measures alongside advanced computational techniques.

The system is intended to connect simulation, trading, risk analysis, and research-oriented methods into a coherent workflow. Human users remain responsible for interpreting system outputs; QuantSim is intended to support decision-making rather than replace it.

1.5 Project Objectives

The objectives of QuantSim are:

1. To provide a reliable market-data ingestion and historical replay pipeline.
2. To implement configurable stochastic market simulation and controlled stress scenarios.
3. To develop an event-driven trading engine capable of processing orders, simulating execution, incorporating transaction costs and slippage, and updating portfolio state.
4. To provide portfolio performance and risk analytics.
5. To investigate the use of Topological Data Analysis for identifying structural changes in financial markets.
6. To investigate machine-learning approaches for volatility forecasting, market-regime classification, and portfolio risk assessment.
7. To provide an integrated dashboard for market, trading, portfolio, and risk analysis.
8. To provide explainable analytical outputs for important ML-based risk alerts.

9. To evaluate the system using quantitative metrics, baseline comparisons, controlled scenarios, and failure cases.

1.6 Project Scope

1.6.1 Core Scope

The committed core scope of QuantSim consists of:

- Market simulation and historical replay.
- Event-driven trading and portfolio-state management.
- Portfolio risk analytics.
- Interactive dashboard functionality.

The system also includes TDA-based market-structure analysis, machine learning for risk-oriented analysis, and controlled stress scenarios as major analytical components.

1.6.2 Extended Scope

Extended functionality includes:

- Financial-news analysis using NLP.
- Explainability mechanisms for machine-learning risk alerts.
- Additional analytical capabilities that can be implemented without compromising the core system.

1.6.3 Out of Scope

QuantSim will not:

- Execute real financial transactions.
- Provide investment advice.
- Function as a brokerage platform.

- Attempt to reproduce a production-grade exchange.
- Attempt to implement unrestricted high-frequency trading.
- Support every financial asset class in the initial implementation.

The initial asset class is equities. The system distinguishes between an asset, such as a stock, and a trade, which is an executed transaction involving that asset.

1.7 Project Modules

1.7.1 Market Data Ingestion

This module acquires historical market data, validates and normalizes the data, and stores it in a form accessible to downstream modules.

1.7.2 Database and Data Management

This module provides persistent storage for market observations, portfolio state, orders, trades, and other system information. The design considers time-series storage, indexing, temporal aggregation, and concurrent access.

1.7.3 Market Simulation

The market simulation module replays historical market conditions and generates controlled synthetic market trajectories using supported stochastic models.

1.7.4 Event-Driven Trading Engine

The trading engine processes market events and strategy actions, validates orders, simulates execution, applies transaction costs and slippage, records trades, and updates portfolio state.

1.7.5 Portfolio Risk Analytics

The risk-analysis layer calculates financial performance and risk measures and incorporates outputs from advanced analytical components.

1.7.6 Topological Analysis

The TDA pipeline transforms financial relationships into topological representations and extracts persistence-based features for market-structure analysis.

1.7.7 Machine Learning

The machine-learning layer is intended to support volatility forecasting, market-regime classification, and risk prediction.

1.7.8 Risk and Alert Engine

The alert engine aggregates analytical outputs and identifies conditions requiring user attention.

1.7.9 Analytics Dashboard

The dashboard provides visual access to simulation, trading, portfolio, risk, and analytical information.

1.7.10 Explainability Layer

The extended explainability layer identifies factors contributing to important ML-based risk assessments using attribution techniques and structured explanations.

1.8 Stakeholders and User Classes

1.8.1 Financial Analysts

Financial analysts require useful representations of portfolio performance and risk. Their primary concern is dashboard utility and the ability to inspect

risk conditions and their contributing factors.

1.8.2 Quantitative Researchers

Quantitative researchers require access to financial data, simulation environments, mathematical analysis, and machine-learning tools in a common experimental environment.

1.8.3 Day Traders and Strategy Developers

Strategy developers require ways to evaluate trading strategies against historical and deliberately constructed market conditions without exposing real capital to those scenarios.

1.8.4 Faculty and Project Evaluators

The supervisor and faculty panel are responsible for evaluating the technical validity, complexity, feasibility, implementation quality, and evidence supporting the project.

1.9 Success Criteria

QuantSim will be considered successful when it can demonstrate a complete path from market input to portfolio analysis.

The primary success criteria are:

1. Historical market data can be acquired, validated, stored, and replayed reliably.
2. The simulation subsystem can produce or replay market conditions suitable for strategy evaluation.
3. The trading engine can process defined orders and update portfolio state while accounting for transaction costs and slippage.
4. Portfolio performance and risk metrics can be calculated consistently.

5. The TDA pipeline can transform representative financial data into persistence-based analytical features.
6. Machine-learning models can be evaluated against appropriate baselines.
7. The dashboard can present the resulting analytical outputs coherently.
8. Important requirements can be demonstrated through observable testing evidence.

1.10 Constraints and Assumptions

QuantSim is constrained by the time, computing resources, and scope of an undergraduate Final Year Project. The system therefore prioritizes a coherent core system over unrestricted feature expansion.

The project assumes access to suitable historical market data through free or academically accessible sources.

The stochastic simulation models are abstractions of financial-market behavior and are not intended to reproduce all mechanisms of real-world exchanges.

The initial implementation focuses on equities.

Machine-learning and TDA outputs will be treated as analytical signals that require empirical validation rather than guaranteed predictions of future market behavior.

1.11 Ethical, Legal and DEI Considerations

QuantSim is intended as a research and decision-support platform. It will not execute real transactions or provide investment advice.

All external financial-data sources and software dependencies must be used according to their applicable terms and licenses. Free or academically accessible data sources and permissively licensed software will be preferred where possible.

The system should preserve analytical auditability by maintaining sufficient information about the inputs and processing that produced an output.

Dashboard design should follow accessibility-oriented principles so that important information is not conveyed solely through color or complex visualizations.

2 Research and Existing Solutions

2.1 Research Method / Approach

The research process for QuantSim combines domain study, literature review, technical investigation, and comparison of existing systems.

The initial stage focused on financial concepts required for the project, including returns, volatility, correlation, drawdown, risk-adjusted performance, market regimes, and trading execution.

The second stage investigated existing quantitative-finance platforms to identify established functionality and determine an appropriate project boundary.

The third research stream examined the use of Topological Data Analysis in financial time series, including topology-based features, persistence-based measures, and regime-identification applications.

Technical investigation was also conducted into financial-data APIs, time-series databases, market simulation, machine-learning approaches, software architecture, and explainability.

The purpose of this investigation was to select technically defensible approaches rather than maximize the number of technologies incorporated into the project.

2.2 Existing Solutions / Products

The project examined several established financial systems, including SimCorp One, QuantConnect, QuantRocket, and AlgoTrader.

QuantConnect is strongly oriented toward algorithmic strategy development, backtesting, and deployment. QuantRocket provides a research and backtesting environment centered around quantitative workflows.

SimCorp One and AlgoTrader operate at a broader institutional level, with capabilities related to financial operations, trading infrastructure, and portfolio management.

FinRL was also investigated because of its overlap with market environments, trading simulation, portfolio management, backtesting, and algorithmic trading.

For TDA, the project distinguishes between financial applications of topological methods and software libraries that implement TDA algorithms. Libraries such as GUDHI are computational libraries rather than financial analysis platforms and are therefore treated as implementation dependencies rather than direct competing products.

2.3 Comparison of Existing Solutions

The comparison focuses on project-relevant capabilities rather than individual features.

Table 2: Comparison Criteria for Existing Solutions

Capability	**Relevance to QuantSim**
Historical market replay	Core
Configurable market simulation	Core
Controlled stress scenarios	Core/Analytical
Trading and portfolio simulation	Core
Portfolio risk analytics	Core
Machine-learning analysis	Analytical
TDA-based market analysis	Research Component
Explainable risk analysis	Extended
Integrated research workflow	Core

Existing platforms already provide substantial functionality for trading, backtesting, portfolio management, and risk analysis. QuantSim therefore does not claim these capabilities as individually novel.

The project's distinction is the combination of controlled simulation, portfolio-risk analysis, topology-based market-structure analysis, and explainability in a unified research environment.

2.4 Limitations and Gaps

The identified gap is a system-design gap rather than a claim that existing platforms cannot perform individual tasks.

Existing systems generally emphasize algorithmic trading, backtesting, deployment, or institutional financial workflows. QuantSim emphasizes controlled experimentation with market conditions and analysis of the resulting portfolio behavior.

The project also investigates the operational use of TDA as part of an integrated financial-risk workflow rather than as an isolated research experiment.

A further focus is the explainability of combined risk signals, allowing users to investigate why risk has been assessed as elevated rather than receiving only a single numerical value.

2.5 Proposed Solution

QuantSim provides a connected simulation and analytical pipeline.

Historical market data or generated market conditions enter the simulation environment. The trading engine executes a selected strategy against the evolving market state. The resulting portfolio state is passed to the risk and analytical layers.

Conventional risk metrics are combined with TDA-based market-structure analysis and machine-learning outputs. The alert engine identifies important conditions and presents them through the dashboard.

This architecture allows the same experiment to be examined from multiple analytical perspectives.

2.6 Complex Computing Problem

The central Complex Computing Problem in QuantSim is the design and integration of a system capable of representing evolving financial-market conditions, processing event-driven trading activity, maintaining consistent portfolio state, and applying multiple computationally intensive analytical methods to the resulting data.

The complexity arises from the interaction of these components.

Market simulation requires assumptions about stochastic temporal behavior. Trading requires correct event ordering and financial accounting. Risk analysis requires consistent portfolio state and protection against look-ahead bias. TDA requires transformation of financial relationships into mathematical structures before persistence can be extracted. Machine learning introduces additional requirements for feature engineering, evaluation, and generalization.

These requirements must operate as one system rather than as isolated algorithms.

2.7 Technical Challenges

The major technical challenges include:

- Acquiring and validating reliable financial market data.
- Maintaining temporal consistency across market replay, simulation, and trading.
- Designing a realistic but computationally manageable market simulation.
- Correctly processing event-driven trading and portfolio updates.
- Constructing meaningful topological representations of financial relationships.
- Designing machine-learning pipelines that avoid data leakage and overfitting.
- Integrating multiple analytical services while preserving system consistency.
- Providing interpretable outputs for complex analytical results.

2.8 Research Findings and Design Decisions

Research into financial indicators reinforced the use of returns, volatility, correlation, drawdown, and related measures as foundational risk features.

Research into existing trading frameworks showed that QuantSim should not attempt to reproduce a general-purpose trading platform or reinforcement learning framework. The project instead emphasizes controlled simulation, portfolio-risk analysis, and integration of advanced analytical methods.

Research into TDA established the basis for deriving market relationships, constructing distance representations, computing topological complexes, and

extracting persistence-based features.

Technical investigation into time-series storage influenced the design toward temporal data management, indexed observations, aggregation, and separation between raw market data and derived analytical information.

Machine-learning research also influenced model selection. Tree-based methods are suitable for structured financial features, recurrent methods can be considered for sequential patterns, and transformer-based approaches are more appropriate to the extended financial-NLP component than to the initial core risk pipeline.

3 Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The proposal-defense feedback emphasized that QuantSim contained substantial technical scope and therefore required careful prioritization.

The team was advised to strengthen its understanding of stock-market dynamics and financial indicators, investigate APIs and data sources for historical and pseudo-real-time market data, understand the internal mechanisms and use cases of proposed machine-learning approaches, and begin dashboard development early rather than postponing it until the final stage.

Following this feedback, the team refined the project into a core system of market simulation, trading, portfolio-risk analysis, and dashboard functionality, supported by TDA, machine learning, and selected extended analytical components.

3.2 Proposal Action Register

Table 3: Proposal Action Register

No.	Feedback / Required Change	Action Taken	Status	Evidence
1	Project scope was considered broad and required refinement.	Separated the project into a coherent core pipeline and extended functionality.	Addressed	Revised scope
2	Strengthen financial-domain knowledge.	Studied returns, volatility, correlation, drawdown, market regimes, and trading concepts.	Addressed / Ongoing	Research notes
3	Investigate market-data APIs and historical data.	Researched data acquisition approaches and made data ingestion an Iteration 1 foundation.	Addressed / Ongoing	Data-ingestion plan
4	Justify proposed ML methods.	Studied XGBoost, Random Forest, LSTM, and Transformer use cases.	Addressed	Technical research
5	Develop dashboard incrementally.	Dashboard development was distributed across the iteration plan.	Addressed	Iteration plan
6	Investigate financial time-series database design.	Studied time-series storage, aggregation, indexing, and concurrent access.	Addressed / Ongoing	Data architecture research

3.3 Iteration Strategy

QuantSim follows an iterative development strategy in which each iteration produces a demonstrable increment rather than postponing implementation until the end of FYP-1.

Iteration 1 establishes the data and system foundation. Iteration 2 develops the stochastic simulator, trading engine, and TDA pipeline. Iteration

3 introduces machine-learning analysis and alerting. Iteration 4 focuses on integration, dashboard refinement, testing, and selected extended modules.

3.4 Iteration 1 Plan

Iteration 1 establishes the technical foundations required by all subsequent components.

The main objectives are:

- Market-data acquisition and validation.
- Initial database-schema implementation.
- Continuous integration and development infrastructure.
- Authentication and protected API access.

The expected deliverables include a functional data-ingestion path, initial persistent storage, the project API foundation, authentication, and development-quality infrastructure.

3.5 Iteration 2 Plan

Iteration 2 focuses on the core analytical infrastructure.

The selected work includes:

- Stochastic market simulation.
- Event-driven trading engine.
- Topological Analysis pipeline.

The expected outcome is a working path from market conditions through trading to portfolio and analytical outputs.

3.6 Work Breakdown and Responsibilities

The team is organized around meaningful technical ownership rather than separate independent mini-projects.

The work spans:

- Market-data and database infrastructure.
- Market simulation.
- Trading and portfolio processing.
- TDA analysis.
- Machine-learning and risk analysis.
- Dashboard and integration.
- Shared testing, security, documentation, and quality responsibilities.

3.7 Progress Against Plan

The project is being tracked against the approved iteration and milestone plan.

Table 4: Progress Against Planned Milestones

Milestone	**Target**	**Status**	**Evidence**
Market-data ingestion	*[Date]*	*[Status]*	*[Evidence]*
Database schema	*[Date]*	*[Status]*	*[Evidence]*
CI setup	*[Date]*	*[Status]*	*[Evidence]*
Authentication	*[Date]*	*[Status]*	*[Evidence]*

3.8 Delays, Deviations and Recovery Plan

Any deviations from the original milestone plan will be recorded together with their cause, technical impact, revised target, and recovery action.

The primary recovery principle is to preserve the core end-to-end system rather than attempting to protect every optional feature. Extended functionality may therefore be deferred where necessary to protect the delivery of the committed scope.

4 Software Requirements

4.1 User Interactions / Use Cases

The primary user interacts with QuantSim through a web-based dashboard and supporting API workflow.

The principal interactions include:

- Selecting or acquiring market data.
- Configuring a market replay or simulation.
- Configuring or selecting a trading strategy.
- Executing a simulation.
- Monitoring orders, trades, and portfolio state.
- Inspecting portfolio performance and risk metrics.
- Inspecting TDA-based market-structure outputs.
- Reviewing machine-learning risk assessments.
- Reviewing alerts and explanations.

4.2 Use Case Diagram

Figure 1: QuantSim Use Case Diagram

4.3 Detailed Use Cases

4.3.1 Run Historical Market Replay

The user selects a historical dataset and specifies an applicable time interval and simulation configuration. The system retrieves the required observations and processes them in temporal order.

The trading strategy receives the corresponding market information and produces orders. Executions and portfolio-state changes are recorded for later analysis.

4.3.2 Run Simulated Market Scenario

The user selects a supported simulation model and configures its parameters. The system initializes the model and generates the market trajectory over the specified time horizon.

The generated events are passed to the trading engine, which updates the portfolio state as execution occurs.

4.3.3 Analyze Portfolio Risk

After or during a simulation, the user requests portfolio-risk information. The system calculates the configured financial metrics and presents the results through the dashboard.

4.3.4 Inspect Structural Market Risk

The user requests topology-based analysis over a selected period. The system constructs relationships among assets, computes the selected topological representation, and derives persistence-based analytical features.

4.4 Functional Requirements

4.4.1 Market Data

- **FR-01:** The system shall acquire historical market data from supported data sources.
- **FR-02:** The system shall validate and normalize acquired market data.
- **FR-03:** The system shall retrieve market observations by asset and time interval.

4.4.2 Market Simulation

- **FR-04:** The system shall replay historical market observations in temporal order.
- **FR-05:** The system shall generate market observations using supported stochastic models.
- **FR-06:** The system shall support defined adverse-market scenarios within the project scope.

4.4.3 Trading and Portfolio

- **FR-07:** The system shall accept defined trading orders.
- **FR-08:** The system shall validate orders against the simulated market and portfolio state.
- **FR-09:** The system shall simulate execution while applying configured transaction costs and slippage.
- **FR-10:** The system shall update positions, cash, and portfolio performance after execution.

4.4.4 Risk and Analysis

- **FR-11:** The system shall calculate defined portfolio performance and risk measures.
- **FR-12:** The system shall perform topology-based analysis on supported financial data.
- **FR-13:** The system shall produce machine-learning-based risk or market-regime outputs for supported scenarios.
- **FR-14:** The system shall generate risk alerts according to configured decision logic.

4.4.5 Visualization

- **FR-15:** The system shall display simulation, trading, portfolio, and risk information.
- **FR-16:** The system shall allow users to inspect the analytical evidence associated with important alerts.

4.5 Non-Functional Requirements

4.5.1 Reliability

The system shall preserve consistency between market events, executed trades, and portfolio state. Failures in analytical components shall not silently corrupt stored market or portfolio information.

4.5.2 Usability

The dashboard shall present important market and portfolio information in a form understandable to the target user classes. Important alerts shall not depend exclusively on color.

4.5.3 Performance

Performance requirements shall be defined using measured results from the implementation.

4.5.4 Security

Protected system resources shall require authentication. Sensitive credentials shall not be stored in source control, and access to protected operations shall be restricted appropriately.

4.6 Acceptance Criteria

Important requirements shall be accepted using observable evidence.

For example, market-data ingestion is accepted when representative data can be acquired, validated, persisted, and retrieved without corruption.

The trading engine is accepted when a defined sequence of market events and orders produces the expected portfolio state, including configured costs and slippage.

The TDA pipeline is accepted when representative data can be transformed through the specified pipeline and produce reproducible analytical outputs.

4.7 Requirements Traceability

Important requirements will be traced to the corresponding modules, design components, implementation artefacts, and test cases.

5 System Design

5.1 System Architecture

QuantSim follows a layered system architecture intended to separate presentation, API access, analytical services, and persistent data.

The dashboard communicates with backend services through APIs. The backend contains the primary business and analytical responsibilities, including market data, simulation, trading, machine learning, risk, and explainability.

Persistent storage contains market observations, user-related information, orders, trades, portfolio state, and analytical artefacts as required.

Figure 2: QuantSim System Architecture

5.2 Architectural Components

The primary components are:

- Dashboard / presentation layer.

- API and gateway layer.
- Market-data service.
- Simulation service.
- Trading service.
- Machine-learning service.
- Risk-analysis service.
- Explainability service.
- Persistent data layer.

5.3 Interfaces and Dependencies

The components exchange structured representations of market state, orders, trades, portfolio state, and analytical outputs through defined interfaces.

External dependencies include:

- Financial market-data providers.
- Machine-learning libraries and frameworks.
- TDA libraries.
- Database and time-series technologies.
- Supporting software-development infrastructure.

Each dependency must be documented with its purpose and applicable license.

5.4 Architecture Decisions and Rationale

The separation of services is intended to prevent analytical components from being tightly coupled to the dashboard.

The data layer is separated from analytical logic so that multiple modules can operate on common market information.

The storage design prioritizes temporal access patterns because financial observations are naturally indexed by time and asset.

Asynchronous communication may be used where it provides genuine workload separation; infrastructure should not be introduced solely for architectural complexity.

5.5 Design Evolution

The project architecture evolved following proposal feedback and subsequent technical investigation.

The initial proposal included a broad range of simulation, trading, machine-learning, TDA, explainability, dashboard, and infrastructure components. The project was subsequently refined into a clear core pipeline with extended features separated from the core.

5.6 Domain Model

The domain model distinguishes between the financial instruments being traded and the transactions performed on them.

A financial **asset** represents the instrument being simulated. In the initial implementation, the asset class is equities.

An **order** represents a request to buy or sell an asset.

A **trade** represents an executed transaction resulting from an order.

A **position** represents the amount of an asset currently held by the portfolio.

A **portfolio** represents the collection of positions, cash, and associated performance information.

A **market event** represents information that advances the simulated market state and may trigger trading logic.

5.7 Data Design

Market observations are represented as temporal data identified by an asset and timestamp.

The database design is intended to retain raw observations as the source of truth while supporting derived temporal resolutions through aggregation.

Important entities include:

- Assets.
- Market observations.
- Users.
- Portfolios.
- Positions.
- Orders.
- Trades.
- Simulation runs.
- Risk and analytical outputs.

Figure 3: Initial QuantSim Database Design

5.8 Process / Behavior Design

The primary processing flow is:

Market Data → Market State → Strategy → Order → Execution → Portfolio Update → Risk An

Market events must be processed in temporal order.

Where forecasting or predictive analysis is evaluated, only information available at the corresponding simulated time may be used, preventing look-ahead bias.

Figure 4: QuantSim End-to-End Processing Flow

6 Iteration 1: Design and Implementation

6.1 Iteration Objectives

Iteration 1 establishes the technical foundation required for later development.

The objectives are:

- Implement market-data acquisition and validation.
- Establish the initial database schema and persistence layer.
- Establish repository and CI infrastructure.
- Implement authentication and protected API access.

6.2 Selected Use Cases / Features

The selected Iteration 1 use cases and features are:

1. Acquire supported historical market data.
2. Validate and normalize the acquired data.
3. Store market data in the project database.
4. Retrieve market data through the backend.
5. Establish authentication.
6. Establish automated development checks.

6.3 Design

The Iteration 1 design separates data acquisition, validation, and persistence.

Figure 5: Iteration 1 Design

6.4 Implementation

The implementation of Iteration 1 consists of the market-data acquisition pipeline, data validation and normalization functions, initial database entities, backend interfaces, authentication, and automated development checks.

6.5 Completed Features / Module

The actual completion state of each feature should be recorded here.

Table 5: Iteration 1 Feature Status

Feature	**Status**	**Evidence**
Market-data integration	*[Status]*	*[Evidence]*
Data validation	*[Status]*	*[Evidence]*
Database schema	*[Status]*	*[Evidence]*
API foundation	*[Status]*	*[Evidence]*
Authentication	*[Status]*	*[Evidence]*
CI pipeline	*Completed*	*Github Testing*

6.6 Technical Implementation Details

This section should document actual implementation artefacts, including:

- Source files and modules.
- Important classes and functions.
- Database tables or hypertables.
- API endpoints.

- Validation mechanisms.
- Authentication implementation.
- CI configuration.
- Important configuration parameters.

6.7 Algorithms and Technical Decisions

The market-data pipeline transforms externally obtained records into the normalized internal market representation.

Validation identifies invalid timestamps, missing fields, duplicate observations, and other malformed records according to the data source.

The database design uses temporal indexing and structured data representation to support future simulation and analytical workloads.

6.8 Tools, Frameworks and Technologies

The actual Iteration 1 technologies should be documented here, including versions where appropriate.

Table 6: Iteration 1 Technologies

Technology	**Purpose**	**Version**
[Technology]	*[Purpose]*	*[Version]*
[Technology]	*[Purpose]*	*[Version]*
[Technology]	*[Purpose]*	*[Version]*

6.9 Demonstration Evidence

The Iteration 1 demonstration should provide evidence of:

- Successful market-data acquisition.
- Validated data entering the persistence layer.

- Successful database retrieval.
- Successful authentication.
- Successful automated CI checks.

Figure 6: Iteration 1 Demonstration Evidence

6.10 Iteration Outcome

The outcome of Iteration 1 should be described using actual implementation and validation evidence.

The expected outcome is a stable system foundation on which the stochastic market simulator and event-driven trading engine can be built in Iteration 2.

7 Testing and Early Validation

7.1 Testing Strategy

Testing is conducted progressively as system components are implemented.

Iteration 1 testing focuses on data integrity, persistence, API behavior, authentication, and failure handling. Later iterations will extend testing to trading, simulation, TDA, machine learning, and complete end-to-end workflows.

7.2 Test Levels and Techniques

The project uses:

- Unit testing.
- Integration testing.
- System testing.
- Edge-case testing.

- Failure-case testing.
- Validation against mathematical and financial expectations.

7.3 Test Cases

Table 7: Representative Test Cases

ID	Test	Expected Result	Status
TC-01	Valid market-data response	Data is ingested and stored correctly	*[Status]*
TC-02	Duplicate record	Duplicate is detected/handled	*[Status]*
TC-03	Invalid market record	Invalid data is rejected or flagged	*[Status]*
TC-04	Valid data retrieval	Correct observations are returned	*[Status]*
TC-05	Unauthenticated request	Protected endpoint is rejected	*[Status]*
TC-06	Valid authentication	Authorized operation succeeds	*[Status]*

7.4 Test Results

7.5 Acceptance Testing / Validation

Acceptance testing will ultimately assess whether representative users can perform relevant QuantSim tasks successfully.

The final acceptance plan will identify the participants or justified domain proxies, the tasks performed, observed outcomes, issues identified, and acceptance decision.

7.6 Edge and Failure Cases

Initial edge and failure cases include:

- External data source unavailable.
- Malformed provider response.
- Missing market observations.
- Duplicate data.
- Invalid API input.
- Authentication failure.
- Database unavailable.

7.7 Defects and Corrective Actions

7.8 Current Validation Status

8 Project Management and Development Evidence

8.1 Work Completed

The project has completed the proposal-stage research, feasibility investigation, literature review, existing-solution comparison, project scoping, architectural planning, and proof-of-concept work required for proposal approval.

Post-proposal implementation evidence should be summarized here according to the actual reporting cutoff.

8.2 Work Remaining

Remaining work includes completion of the market-data and persistence foundation, stochastic market simulation, event-driven trading, TDA anal-

ysis, machine-learning risk analysis, dashboard integration, testing, evaluation, and selected extended modules.

8.3 Individual Responsibilities

The team members hold meaningful technical responsibilities distributed across the system.

Table 8: Team Responsibilities

Member	**Primary Responsibilities**
Hamza Kaleem Sheikh	*[Insert responsibilities]*
Ali Mazhar	*[Insert responsibilities]*
Afnan Rizwan	*[Insert responsibilities]*

8.4 Contribution of Each Team Member

Each member's contribution should include technical decisions, implementation, validation, and integration responsibilities.

8.5 Repository and Version Control Evidence

The project repository maintains source code, configuration, documentation, and version history.

8.6 Milestones and Commit Evidence

The team tracks milestones against the iteration plan.

8.7 Tools and Tool-Assisted Development

The project uses standard software-development tools for implementation, testing, collaboration, and documentation.

Any AI-assisted development must be disclosed according to applicable FYP requirements. The team remains responsible for understanding, modifying, testing, and validating all project code.

8.8 External Components and Dependencies

External components include market-data sources, software libraries, machine-learning frameworks, TDA implementations, databases, and development infrastructure.

Each important dependency will be documented according to its role, license, and degree of contribution to the system.

8.9 Risks and Mitigation

Major risks include:

- Insufficient or inconsistent financial data.
- TDA signals failing to generalize out-of-sample.
- Machine-learning overfitting or distribution shift.
- Integration complexity.
- Schedule pressure.
- Scope expansion.

The primary mitigation strategy is to preserve the core simulation, trading, risk, and dashboard pipeline and defer optional functionality when necessary.

8.10 Security and Privacy Considerations

QuantSim does not require sensitive personal financial information for its core operation.

Authentication credentials must be protected, secrets must not be committed to source control, and protected API resources must require appropriate

authorization.

8.11 Current Limitations

The current system remains subject to limitations imposed by its scope and available resources.

The market simulation is a controlled abstraction and cannot reproduce every mechanism of real financial markets.

TDA outputs require empirical validation and should not automatically be interpreted as reliable predictive signals.

Machine-learning outputs may be affected by overfitting, data limitations, and market-regime changes.

9 Remaining FYP Plan

9.1 Remaining Scope

The remaining committed scope consists of:

- Stochastic market simulation.
- Event-driven trading.
- Portfolio-risk analytics.
- TDA-based market-structure analysis.
- Machine-learning risk analysis.
- Risk-alert functionality.
- Integrated dashboard.

Extended components will be completed only where they do not compromise the core system.

9.2 Remaining Modules and Features

The remaining modules are:

- Stochastic Market Simulator.
- Event-Driven Trading Engine.
- TDA Pipeline.
- Machine-Learning Layer.
- Risk and Alert Engine.
- Analytics Dashboard.
- Explainability and selected extended components.

9.3 Remaining Iterations

9.3.1 Iteration 2

The second iteration will implement the stochastic market simulator, event-driven trading engine, and TDA pipeline.

9.3.2 Iteration 3

The third iteration will implement machine-learning analysis and alert generation.

9.3.3 Iteration 4

The fourth iteration will focus on dashboard refinement, integration testing, system-wide evaluation, and selected extended modules.

9.4 Implementation Plan

Development will proceed from foundational data and state management toward higher-level analytical functionality.

The market-data and database infrastructure will provide the foundation for simulation.

The simulation and trading layers will establish a consistent portfolio trajectory.

Risk, TDA, and machine-learning modules will subsequently consume this information.

The dashboard will integrate the outputs of the underlying system into a single user workflow.

9.5 Testing and Validation Plan

Testing will progress from component-level tests to integrated system evaluation.

The trading engine will be evaluated against controlled scenarios with known expected outcomes.

Simulation methods will be checked against their expected statistical and behavioral properties.

The TDA pipeline will be evaluated against representative financial datasets and suitable comparison methods.

Machine-learning models will be evaluated on held-out data using appropriate performance metrics.

The complete system will be evaluated using end-to-end scenarios and appropriate user acceptance activities.

9.6 Deployment Plan

The intended deployment target is a web-based research environment accessed through a dashboard and backend API.

The final deployment environment will be selected according to the computational requirements of the implemented system and the infrastructure

available to the team.

Configuration and setup instructions will be provided so that the final system can be installed and operated independently.

9.7 Tentative Timeline

The project follows the four-iteration plan established during proposal approval.

Table 9: Tentative Development Timeline

Iteration	**Primary Work**
Iteration 1	Data ingestion, database schema, CI, authentication
Iteration 2	Stochastic simulator, trading engine, TDA pipeline
Iteration 3	ML layer, risk and alert engine
Iteration 4	Dashboard refinement, integration testing, extended modules

9.8 Expected Final Deliverables

The expected final deliverables include:

- Working QuantSim platform.
- Market-data ingestion and historical replay.
- Stochastic market simulation.
- Event-driven trading simulation.
- Portfolio performance and risk analytics.
- TDA-based market-structure analysis.
- Machine-learning-based risk analysis.

- Integrated analytical dashboard.
- Explainability functionality where completed.
- Test and evaluation evidence.
- Deployment and setup documentation.
- Complete source code and associated project artefacts.

10 Conclusion

QuantSim addresses the difficulty of safely and systematically evaluating trading strategies and portfolio behavior under changing financial-market conditions. The project provides a controlled environment in which historical markets can be replayed and alternative or adverse market scenarios can be simulated before being passed through an event-driven trading engine.

The research conducted during FYP-1 established that the individual components of the project already exist across financial platforms, research literature, and open-source libraries. The project therefore does not claim that simulation, trading, machine learning, or TDA are individually novel. Instead, the proposed contribution is the integration of controlled market simulation, portfolio-risk analysis, topology-based market-structure analysis, and explainable analytical outputs within one research-oriented environment.

The system qualifies as a Complex Computing Problem because the principal challenge lies in integrating multiple interacting computational and financial requirements. Temporal correctness, stochastic simulation, event-driven execution, portfolio accounting, TDA, machine learning, and system integration all introduce constraints and trade-offs that cannot be reduced to routine software implementation.

The FYP-1 development strategy proceeds iteratively. The first iteration establishes the system and data foundation, after which the stochastic simulator, trading engine, and TDA pipeline form the core analytical infrastructure.

Machine learning, alerting, dashboard integration, and extended components follow in later iterations.

The remaining work will therefore focus not only on implementing functionality, but also on validating the resulting system using measurable evidence, controlled scenarios, appropriate baselines, and explicit failure cases.

A Proposal Action Register

B Detailed Requirements

C Detailed Test Cases

D Repository and Contribution Evidence

E Additional Design Diagrams

F Meeting and Progress Evidence

References

[1] D. Gidea and Y. Katz, *Topological Data Analysis of Financial Time Series*, [Complete bibliographic information to be inserted].

[2] AI4Finance Foundation, *FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance*, [Complete bibliographic information to be inserted].

[3] GUDHI Project, *GUDHI: Geometry Understanding in Higher Dimensions*, [Library/documentation reference].